TP PROGRAMACIÓN ORIENTADA A OBJETOS

1. Presentación

Cada grupo resolverá una situación problemática real a través de una solución de software. El trabajo recorre el ciclo completo que se vio durante la materia: partir de un enunciado de negocio, modelarlo con UML, mejorar el diseño aplicando patrones (SOLID y GRASP) y finalmente construir, probar y documentar un programa en Java.
Se ofrecen tres propuestas distintas (sección 6). Cada grupo elige una. Todo el trabajo debe estar documentado y versionado en un repositorio de GitHub, que permitirá seguir el avance del grupo y el aporte de cada integrante.

2. Objetivos

Al finalizar el trabajo, el grupo será capaz de:
1. Modelar una situación de negocio mediante diagramas UML (casos de uso, clases y secuencia), identificando actores, clases, relaciones, responsabilidades e interacciones. (hechos y conceptos)
2. Fundamentar la aplicación de patrones de diseño (tres de SOLID y cuatro de GRASP), explicando qué problema resuelve cada uno y en qué parte del diseño se ve reflejado. (conceptual)
3. Implementar en Java una aplicación funcional coherente con el modelo, integrando herencia, colecciones, excepciones y entrada/salida de datos, y verificarla con pruebas. (procedimental)
4. Valorar el trabajo colaborativo, la documentación y el control de versiones como prácticas propias del desarrollo profesional de software. (valorativo)

3. Organización general

Aspecto	Definicion
Grupos	2 o 3 integrantes. Se conforman en la primera semana.
Elección	Cada grupo elige una de las tres propuestas y la comunica al docente. No puede haber cambio de propuesta luego de la primera semana.
Etapas	Etapa 1: modelado, patrones y diagrama de clases final. Etapa 2: codificación, puesta a punto y testeo.
Herramientas sugeridas	Diagramas: draw.io, StarUML, PlantUML o similar. Java 17 o superior. Versionado: Git y GitHub.
Defensa	Presentación oral grupal con preguntas individuales a cada integrante.

Cronograma de entrega

- Entrega Etapa 1: 25/10/26
- Entrega Etapa 2: 04/11/26
- Defensa TPO: 11/11/26

4. Consignas

Etapa 1: Modelado y aplicación de patrones

1. **Análisis del enunciado**
   Listar los actores, los requerimientos funcionales y las reglas de negocio. Si algo del enunciado es ambiguo, el grupo debe registrar el supuesto que adopta.
2. **Diagrama de casos de uso**
   Representar los actores y los casos de uso del sistema, con las relaciones de inclusión o extensión que correspondan. Incluir una breve descripción textual de cada caso de uso (actor, precondición, flujo principal, flujos alternativos).
3. **Diagrama de clases inicial**
   Identificar clases, atributos, métodos, visibilidad, multiplicidades y relaciones (asociación, agregación, composición, herencia). Debe incluir al menos una jerarquía de herencia con sobreescritura de métodos.
4. **Diagramas de secuencia**
   Elegir dos casos de uso del diagrama anterior (uno de ellos con un flujo alternativo o de error) y desarrollar para cada uno su diagrama de secuencia, con objetos, mensajes, retornos y fragmentos alt, opt o loop cuando corresponda.
5. **Aplicación de patrones**
   Revisar el diseño y aplicar:
   a. 3 principios SOLID a elección (por ejemplo, responsabilidad única, abierto/cerrado, sustitución de Liskov, segregación de interfaces, inversión de dependencias).
   b. 4 patrones GRASP a elección (por ejemplo, Experto, Creador, Controlador, Bajo acoplamiento, Alta cohesión, Polimorfismo, Fabricación pura, Indirección, Variaciones protegidas).
   Para cada uno se debe documentar: qué problema del diseño detectó el grupo, qué decisión tomó, en qué clases se ve reflejado y qué ventaja aporta. No alcanza con nombrar el patrón: hay que justificarlo.
6. **Diagrama de clases final**
   Rediseñar el diagrama de clases incorporando los cambios derivados de los patrones. Se debe adjuntar una nota que indique qué cambió respecto de la versión inicial y por qué.

Etapa 2: Codificación, puesta a punto y testeo

1. Codificación. Programar en Java las clases del diagrama final. Si durante la codificación el grupo debe apartarse del modelo, tiene que actualizar el diagrama y justificar el cambio.

2. Funcionalidad mínima. Implementar como mínimo los casos de uso definidos en la propuesta elegida (sección 6), incluidos los dos que se modelaron en secuencia, que

deben funcionar tal como fueron diagramados.

3. Requisitos técnicos comunes. Cumplir con los requisitos de la sección 7.

4. Testeo. Diseñar y ejecutar pruebas unitarias sobre las clases con lógica de negocio y un plan de pruebas funcionales (casos de prueba con dato de entrada, resultado esperado y resultado obtenido) para los casos de uso principales.

5. Puesta a punto. Corregir los errores detectados, revisar el manejo de excepciones y verificar que el programa se pueda ejecutar siguiendo únicamente las instrucciones del README.

6. Documentación final. Completar el README y el informe final (sección 8).

5. Repositorio de GitHub y control de versiones

El repositorio es parte de la evaluación. Es el medio por el cual el docente verifica el avance del grupo y el aporte de cada integrante.

Reglas
• Un repositorio por grupo, creado en la semana 1. Se agrega al docente como colaborador

• Todos los integrantes deben realizar commits propios, tanto en la documentación y el modelado como en el código. Un integrante sin commits propios no puede acreditar su participación.

• Los commits deben tener mensajes claros y con sentido (por ejemplo, «Agrega clase Reclamo con validación de estado»). No se aceptan mensajes del tipo «cambios» o «arreglos».

• Se trabaja con ramas: la rama main contiene solo versiones revisadas. Los cambios se integran mediante pull requests revisados por al menos otro integrante.

• El avance debe ser continuo: no se aceptan repositorios cargados en un único día cerca de la fecha de entrega.

• Cada etapa se entrega marcando una versión con un tag (etapa-1, etapa-2).

Estructura sugerida
/docs enunciado elegido, requisitos, informe, plan de pruebas
/docs/uml diagramas (imagen exportada y archivo fuente)
/src código fuente Java
/test pruebas unitarias
README.md descripción, integrantes, instrucciones de ejecución

Etapa 1 - Modelado y aplicación de patrones

PUNTO 1 - ANÁLISIS DEL ENUNCIADO

Actores:

vecinos: titular del lote, familiares, inquilinos y personas autorizadas por el titular.

administrador: se encarga de los reclamos, expensas, empleados y mantenimiento.

personal:

guardias de seguridad (portería, ronda o monitoreo)

empleados de mantenimiento (electricidad, plomería o jardinería).

visitas: visitantes particulares, proveedores y proveedores habituales.

Sistema (actor interno) — dispara: escalado de reclamo a las 24h, cierre automático a los 5 días, cálculo de mora.

Sistemas externos (opcionales): sistema de pagos y cámaras de seguridad.

Requerimientos funcionales:

gestión de vecinos: registrar vecinos, lotes, titulares y personas autorizadas.

reclamos: registrar reclamos, asignarlos a empleados, cambiar su estado, agregar comentarios y consultar su seguimiento.

expensas: generar las expensas mensuales, registrar pagos, calcular intereses por atraso y consultar deudas.

mantenimiento: registrar cámaras y luminarias, informar fallas, generar órdenes de mantenimiento, asignar empleados y registrar reparaciones.

visitas: autorizar visitantes y proveedores, verificar permisos y registrar horarios de entrada y salida.

reportes: consultas reclamos pendientes, lotes con deudas y equipos fuera de servicio.

Reglas de negocio:

reclamos: pueden ser de seguridad, mantenimiento, convivencia o administrativos. Cada uno tiene una prioridad y pasa por distintos estados. solo el titular o alguien autorizado puede reclamar. cada empleado puede tener hasta 5 reclamos en curso. los reclamos resueltos pueden reabrirse dentro de los 5 días.

expensas: vencen el día 10 de cada mes. si no se pagan, se cobra un 3 % mensual de interés. Se permiten pagos parciales y con más de 3 períodos impagos el lote se considera moroso. los pagos no se eliminan, se corrigen.

mantenimiento: las cámaras con fallas tienen prioridad alta. Las luminarias tienen prioridad normal, salvo que fallen más de 3 en el mismo sector. si una cámara lleva más de 48 horas sin funcionar, se avisa al administrador. para cerrar una reparación se debe registrar la fecha, el empleado y el trabajo realizado.

visitas: necesitan autorización previa del vecino, con fecha y horario. Los proveedores habituales pueden tener permisos de hasta 30 días. No se puede registrar otro ingreso sin haber registrado antes la salida.

Escalado automático de reclamos de prioridad alta: si no fue atendido en 24h, el sistema escala al administrador. Lo mencionaron en los supuestos pero no en las reglas.

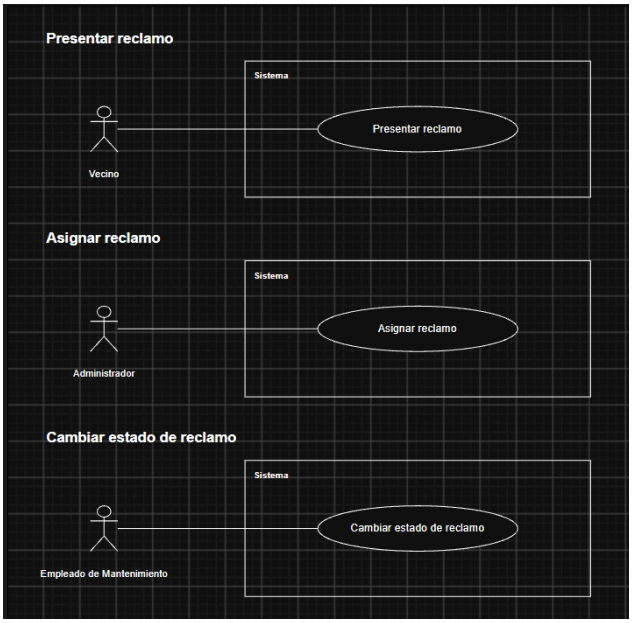
Supuestos:

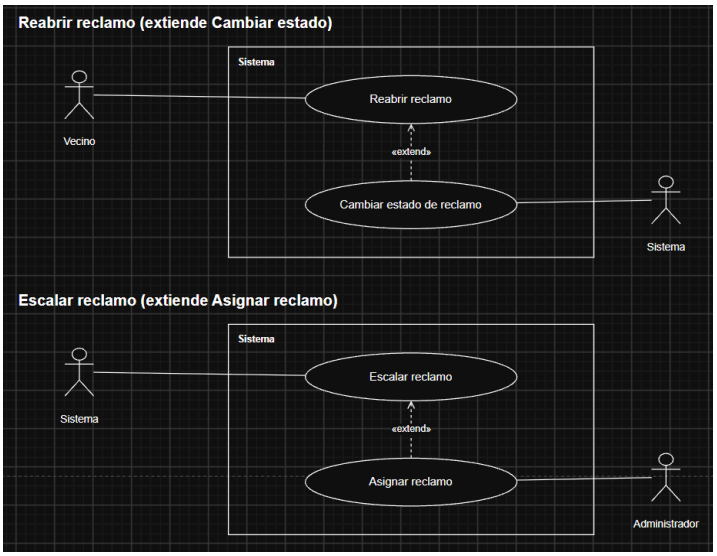
Cada usuario tendrá acceso a distintas funciones según su rol.
Los vecinos podrán consultar sus expensas y reclamos.
El administrador se encargará de registrar los pagos y asignar las tareas.
Los empleados podrán actualizar el estado de sus trabajos.
Los guardias controlarán las visitas y podrán registrar fallas.
Los pagos se registran manualmente.
Las notificaciones se mostrarán dentro del sistema.
Los datos quedarán guardados en archivos para no perder la información.

PUNTO 2 - DIAGRAMA DE CASOS DE USO

CU01 - Registrar y gestionar solicitudes y reclamos

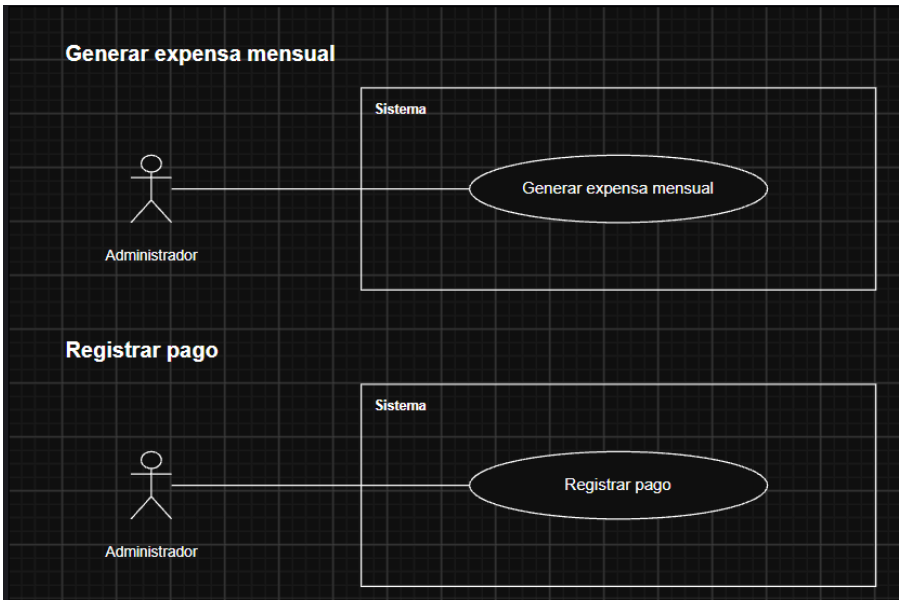
- **Actor:** Vecino (titular o persona autorizada) / Administrador / Empleado de Mantenimiento
- **Precondición:** El vecino tiene que estar autenticado y con un lote activo asociado.
- **Flujo principal:** El vecino carga el reclamo con tipo y descripción; el sistema le asigna prioridad automáticamente y lo deja en estado “abierto”. El administrador lo asigna a un empleado disponible —máximo 5 reclamos en curso por empleado—, que va actualizando el estado hasta resolverlo.
- **Flujo alternativo:** Si pasaron 24 h y un reclamo de prioridad alta sigue sin atención, el sistema lo escala al administrador. Si el vecino reabre un reclamo resuelto dentro de los 5 días, vuelve a “en curso”; pasado ese plazo, queda cerrado y no puede reabrirse.

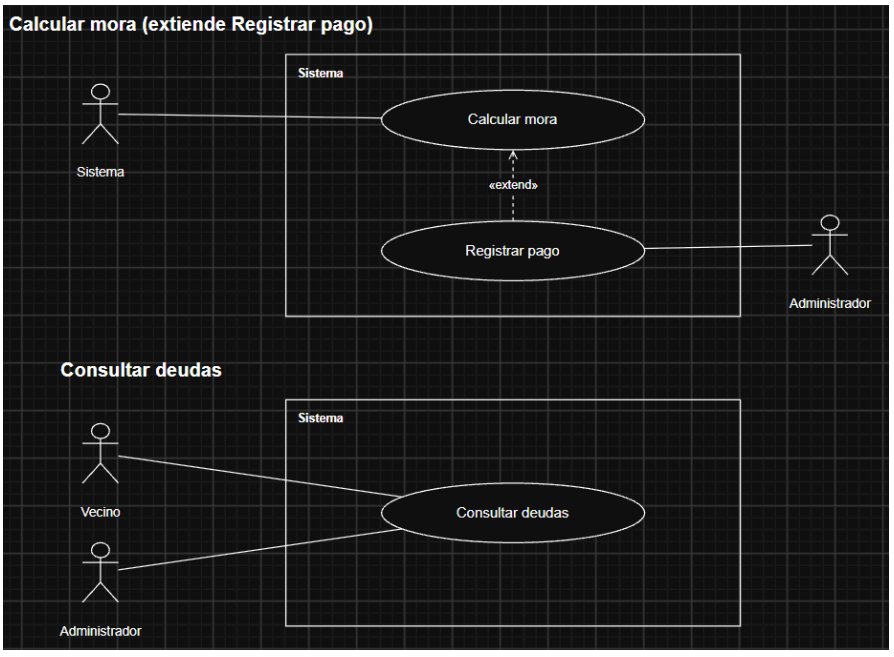




CU02 - Emitir expensas, registrar pagos y calcular intereses por mora

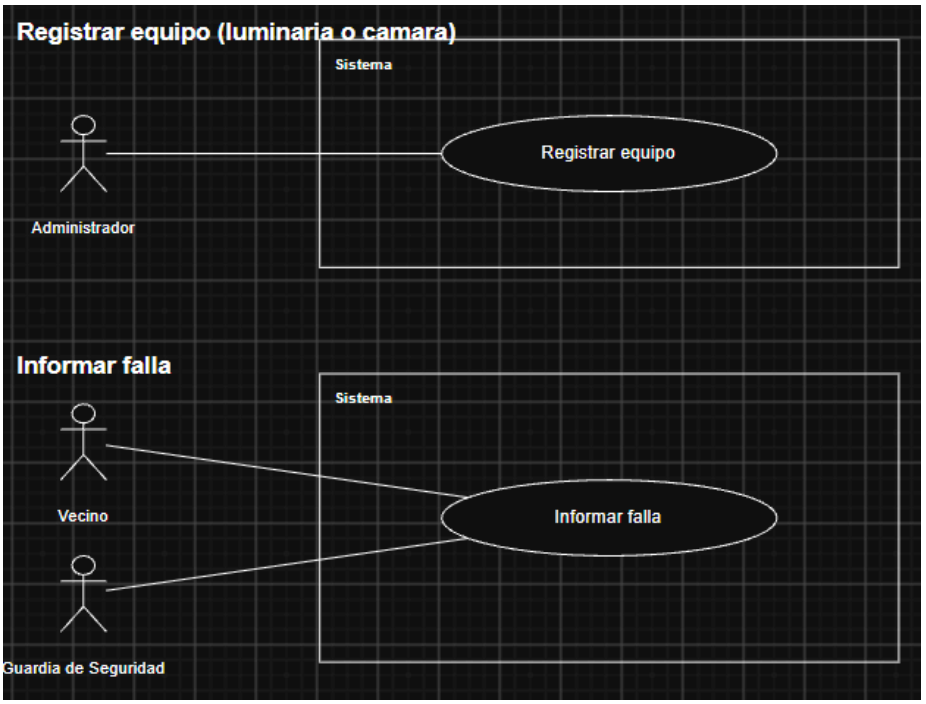
- **Actor:** Administrador / Vecino
- **Precondición:** El presupuesto mensual y los coeficientes de participación de los lotes tienen que estar cargados.
- **Flujo principal:** El administrador genera la expensa de cada lote (presupuesto × coeficiente de participación) con vencimiento el día 10. Los pagos —los registra el vecino o el administrador— se imputan primero a intereses y luego a capital, empezando por el período más antiguo.
- **Flujo alternativo:** Si el pago llega después del vencimiento, el sistema calcula un interés del 3% mensual sobre el saldo impago. Cuando un lote acumula más de 3 períodos sin pagar, se lo marca como moroso y se notifica al titular.

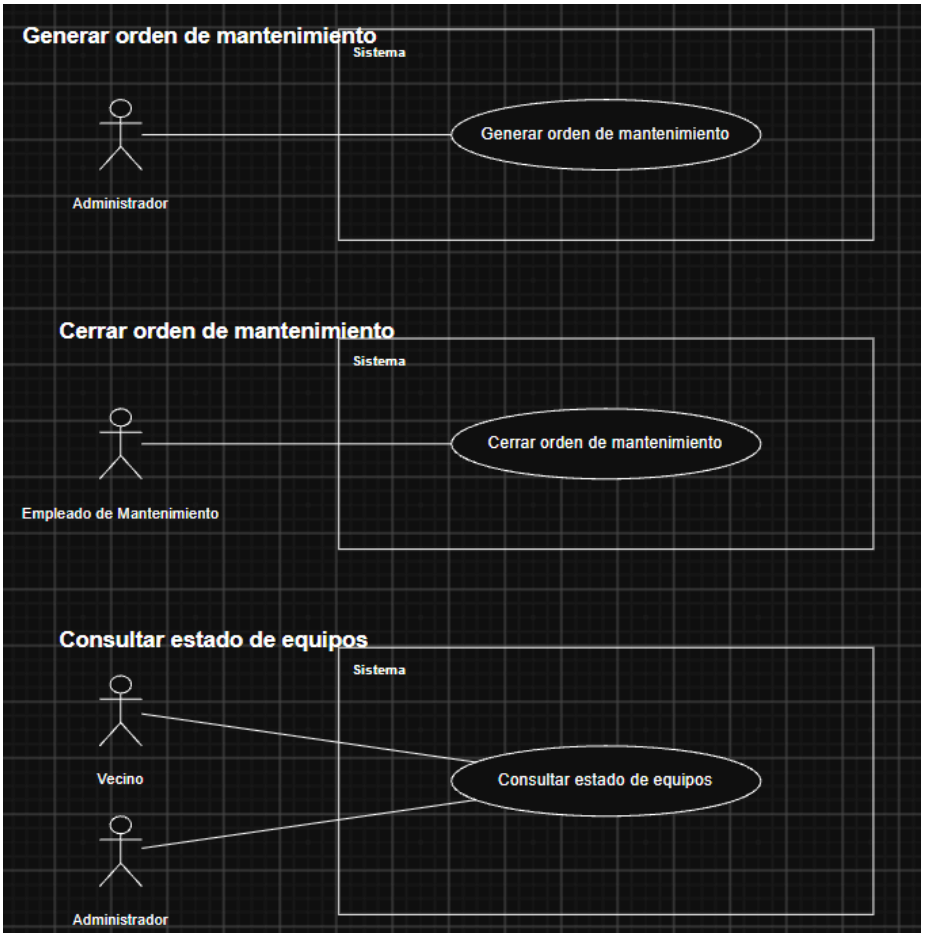




CU03 - Controlar luminarias y cámaras de seguridad

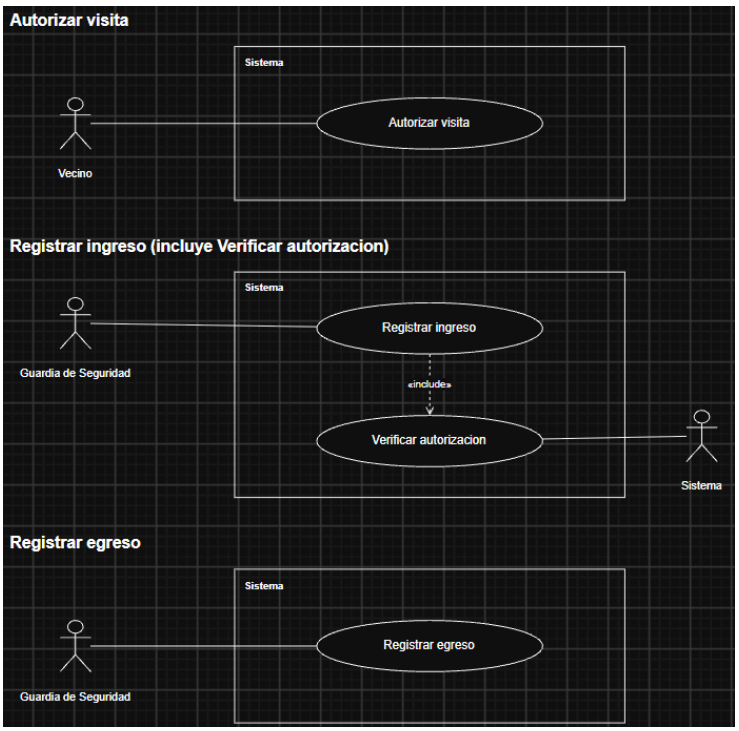
- **Actor:** Administrador / Vecino / Guardia de Seguridad / Empleado de Mantenimiento
- **Precondición:** El equipo (luminaria o cámara) tiene que estar registrado en el sistema.
- **Flujo principal:** Un vecino o guardia reporta la falla; el sistema cambia el estado del equipo y el administrador genera una orden de mantenimiento asignada a un empleado. El empleado la cierra registrando fecha, técnico responsable y descripción del trabajo.
- **Flujo alternativo:** Las fallas de cámara se crean siempre con prioridad alta; las de luminaria son prioridad normal, salvo que fallen más de 3 del mismo sector. Si una cámara está fuera de servicio más de 48 h, el sistema avisa al administrador.





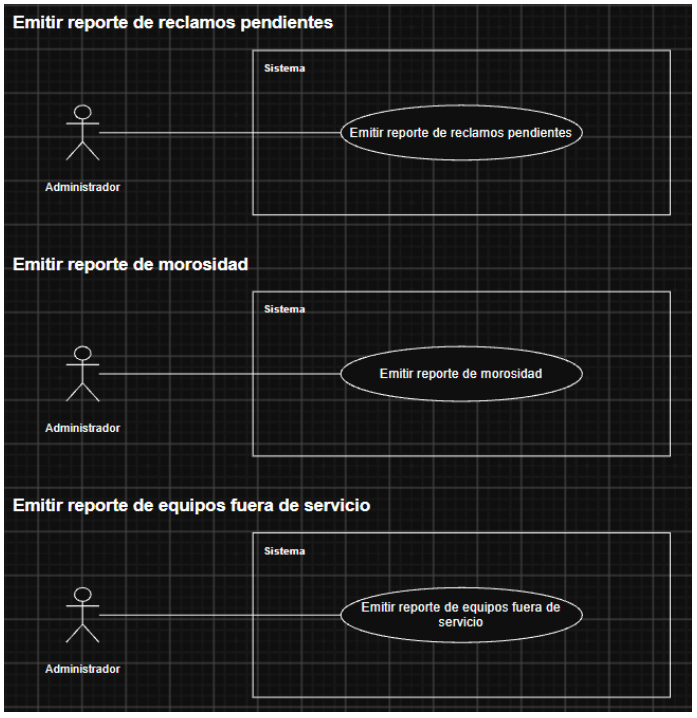
CU04 - Registrar el ingreso de visitas y proveedores con autorización previa

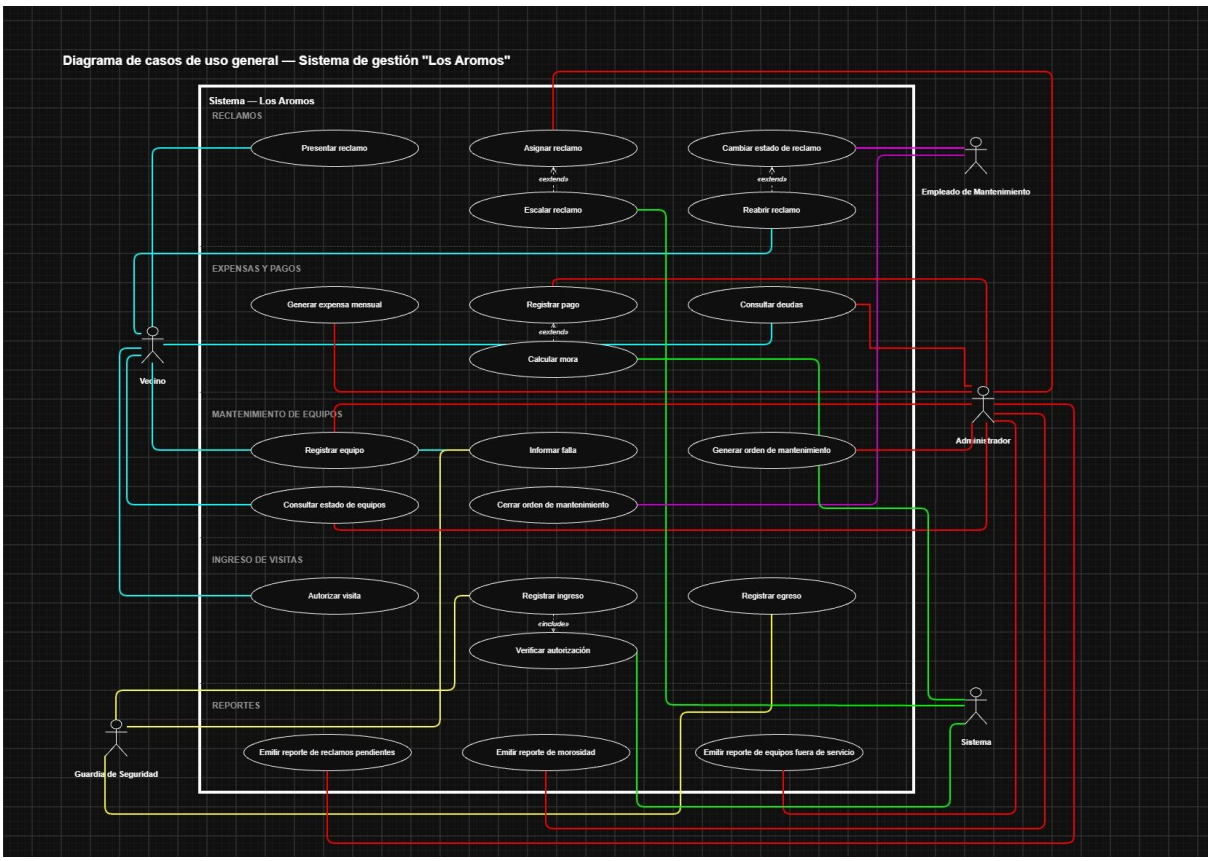
- **Actor:** Vecino / Guardia de Seguridad
- **Precondición:** El visitante o proveedor tiene que estar en portería con su documento.
- **Flujo principal:** El vecino autoriza al visitante con anticipación, indicando fecha y franja horaria. Para proveedores habituales puede generar una autorización recurrente de hasta 30 días. En portería, el guardia verifica que la autorización esté vigente, registra la hora de ingreso y, al retirarse, la de egreso.
- **Flujo alternativo:** Sin autorización vigente, el sistema directamente no habilita el ingreso. Si la persona ya tiene un ingreso abierto sin egreso registrado, cualquier nuevo intento queda bloqueado hasta que se cierre el anterior.



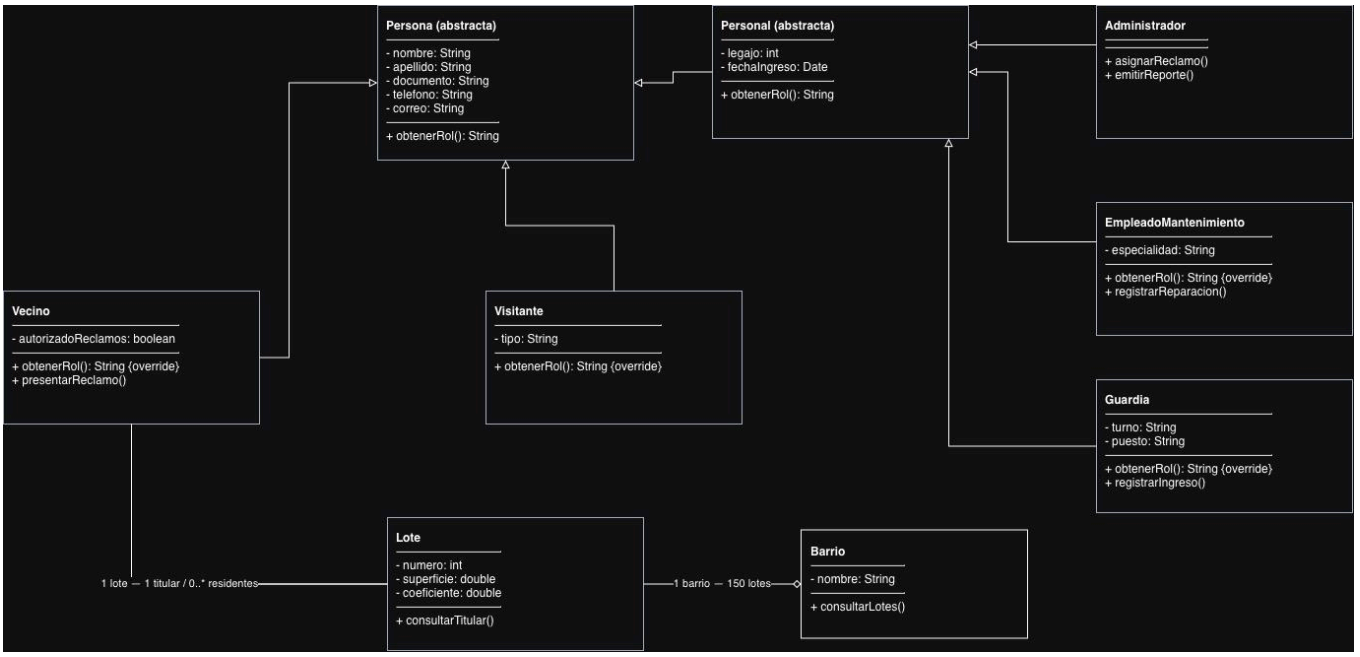
CU05 - Emitir reportes

- **Actor:** Administrador
- **Precondición:** Ninguna.
- **Flujo principal:** El administrador elige el tipo de reporte (reclamos pendientes, morosidad por lote o equipos fuera de servicio) y el sistema genera el listado con los datos actualizados al momento.
- **Flujo alternativo:** Si no hay datos para el reporte elegido, se muestra la lista vacía con un mensaje explicativo.




Diagrama de casos de uso general — Sistema de gestión "Los Aromos"
Sistema — Los Aromos
RECLAMOS
Presentar reclamo
Asignar reclamo
Cambiar estado de reclamo
«extend»
«extend»
Escalar reclamo
Reabrir reclamo
EXPENSAS Y PAGOS
Generar expensa mensual
Registrar pago
Consultar deudas
«extend»
Calcular mora
MANTENIMIENTO DE EQUIPOS
Registrar equipo
Informar falla
Generar orden de mantenimiento
Consultar estado de equipos
Cerrar orden de mantenimiento
INGRESO DE VISITAS
Autorizar visita
Registrar ingreso
Registrar egreso
«include»
Verificar autorización
REPORTES
Emitir reporte de reclamos pendientes
Emitir reporte de morosidad
Emitir reporte de equipos fuera de servicio
Empleado de Mantenimiento
Vecino
Administrador
Sistema
Guardia de Seguridad

PUNTO 3 - DIAGRAMA DE CLASES



1. personas y lotes

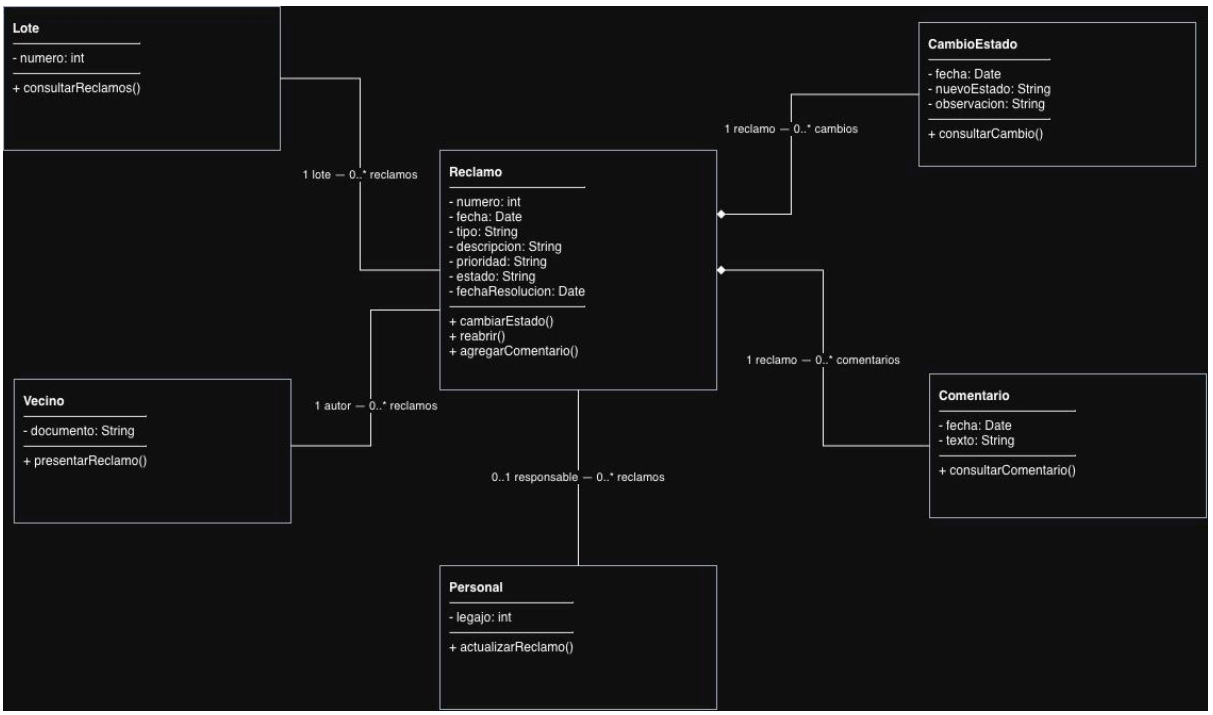
Persona es la clase principal porque todos tienen datos en común, como nombre, apellido y documento.

Vecino, Visitante y Personal heredan de Persona porque son distintos tipos de personas.

Administrador, Guardia y EmpleadoMantenimiento heredan de Personal porque todos son trabajadores del barrio y tienen legajo y fecha de ingreso.

Vecino y Lote tienen una asociación porque cada lote tiene un titular y puede tener otros residentes. Además, una persona puede ser titular de varios lotes.

Barrio y Lote tienen una agregación porque un barrio está formado por varios lotes, pero cada lote tiene sus propios datos.



2. reclamos

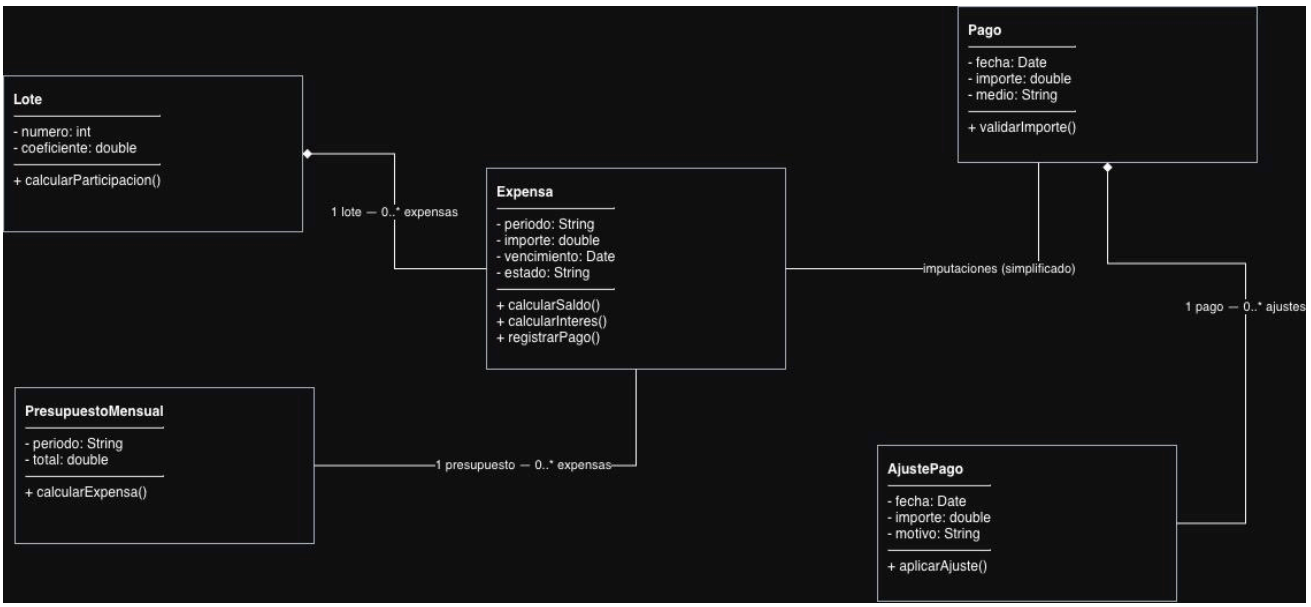
Lote y Reclamo tienen una asociación porque cada reclamo pertenece a un lote.

Vecino y Reclamo tienen una asociación porque un vecino puede presentar varios reclamos.

Personal y Reclamo tienen una asociación porque el administrador asigna un empleado responsable a cada reclamo.

Reclamo y CambioEstado tienen una composición porque los cambios de estado forman parte del historial de ese reclamo.

Reclamo y Comentario también tienen una composición porque los comentarios pertenecen a un reclamo específico.



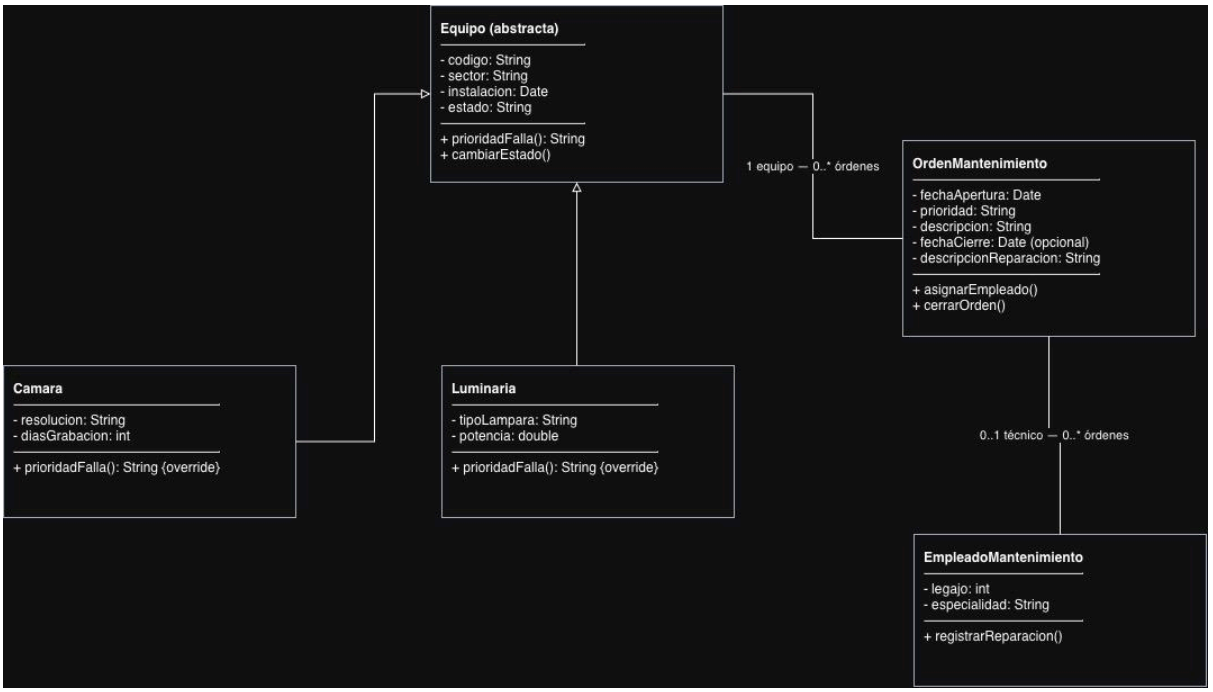
3. expensas y pagos

Lote y Expensa tienen una composición porque las expensas se generan para un lote determinado.

Expensa y Pago tienen una asociación porque los pagos se utilizan para cancelar las deudas. Un pago puede cubrir parte de una expensa o varias deudas anteriores.

Pago y AjustePago tienen una composición porque los ajustes se registran para corregir un pago específico sin eliminarlo.

PresupuestoMensual y Expensa tienen una asociación porque el presupuesto se utiliza para calcular las expensas del mes.



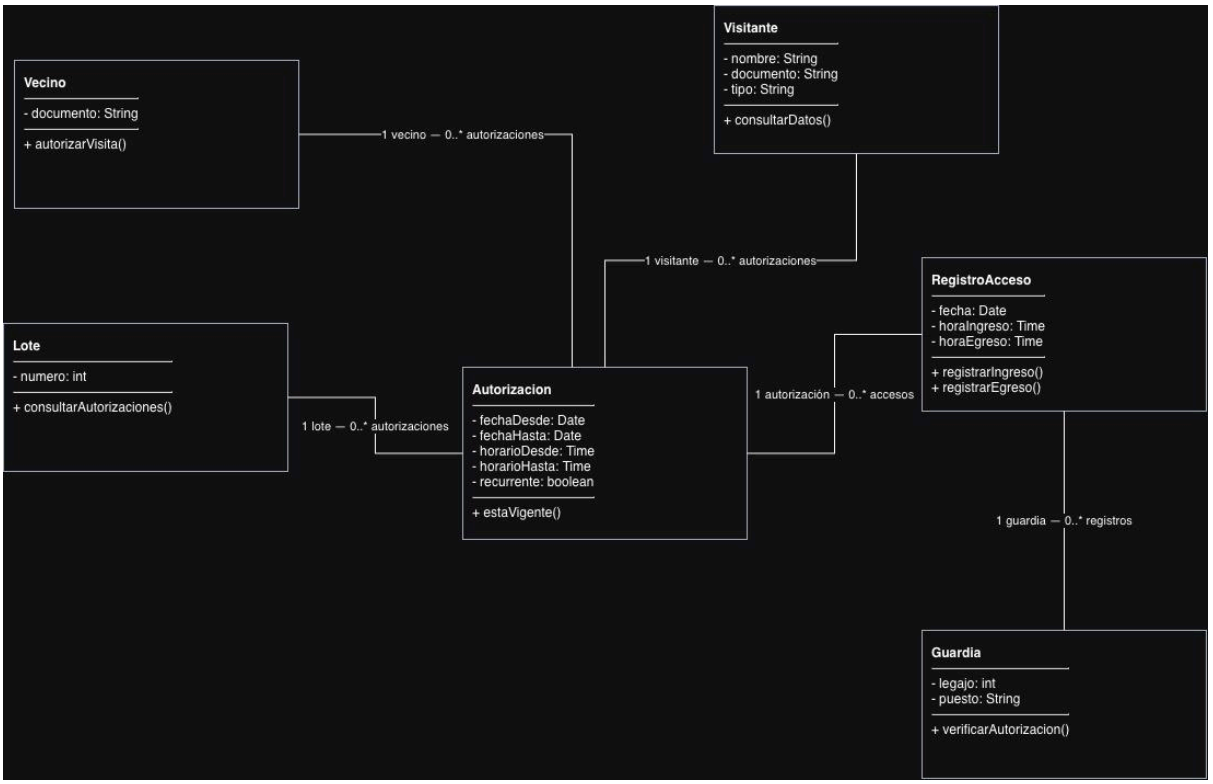
4. mantenimiento

Equipo es la clase principal de Cámara y Luminaria porque ambos tienen código, ubicación, fecha de instalación y estado.

Cámara y Luminaria heredan de Equipo, pero tienen características propias.

Equipo y OrdenMantenimiento tienen una asociación porque un mismo equipo puede necesitar varias reparaciones.

EmpleadoMantenimiento y OrdenMantenimiento tienen una asociación porque los empleados son los encargados de realizar las reparaciones.



5. visitas y autorizaciones

Vecino y Autorizacion tienen una asociación porque un vecino puede autorizar varias visitas.

Lote y Autorizacion tienen una asociación porque cada autorización indica qué lote se va a visitar.

Visitante y Autorizacion tienen una asociación porque una misma persona puede recibir varios permisos de ingreso.

Autorizacion y RegistroAcceso tienen una asociación porque una autorización puede utilizarse para registrar uno o varios ingresos, como sucede con los proveedores habituales.

Guardia y RegistroAcceso tienen una asociación porque el guardia registra las entradas y salidas.

herencia y sobreescritura de métodos

Usamos herencia para evitar repetir datos y funciones que tienen varias clases en común.

Por ejemplo, Persona tiene el método obtenerRol(), pero Vecino y Visitante lo pueden sobrescribir para devolver su propio rol.

También usamos herencia entre Equipo, Cámara y Luminaria.

Ambas heredan el método prioridadFalla(), pero lo sobrescriben porque las cámaras tienen prioridad alta y las luminarias normalmente tienen prioridad normal.

De esta manera cumplimos con el requisito de utilizar herencia y sobreescritura de métodos.

PUNTO 4 - DIAGRAMAS DE SECUENCIA

Diagrama 1 — Consultar deudas

Participantes:
Vecino (actor) → PantallaConsultaDeudas → ControladorExpensas → Lote / Expensa

El vecino selecciona un lote y solicita la consulta de deudas desde la pantalla. La pantalla delega la operación al controlador mediante consultarDeudas(numeroLote), que a su vez le pide el lote al objeto Lote y luego obtiene sus expensas con obtenerExpensas().

Dentro de un fragmento loop (“para cada expensa pendiente”), el controlador le solicita a cada Expensa que calcule su propio interés por mora mediante calcularInteresMora(). Es la propia expensa la que realiza el cálculo, no el controlador, porque es quien tiene los datos necesarios para hacerlo.

Luego el controlador evalúa mediante un mensaje reflexivo (determinarSiEsMoroso()) si el lote superó los 3 períodos impagos. Los retornos (flechas punteadas) devuelven la información hasta mostrarle el resumen de deudas al vecino.

Es el flujo principal sin ramas alternativas.

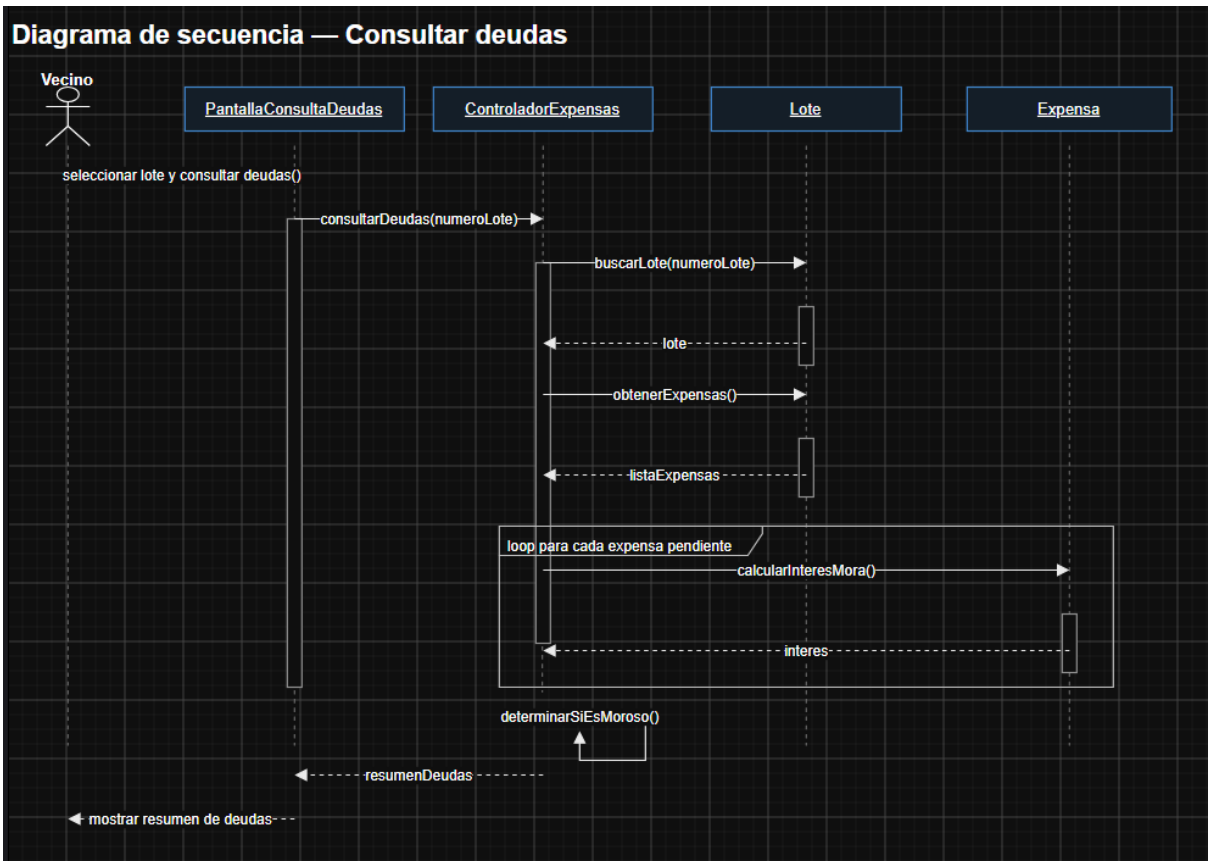


Diagrama 2 — Presentar reclamo

Participantes:
Vecino → PantallaNuevoReclamo → ControladorReclamos → Lote / Reclamo

El vecino completa tipo y descripción y envía el formulario. La pantalla delega la operación al controlador mediante presentarReclamo(vecino, lote, tipo, desc).
El controlador verifica contra Lote si quien reclama es titular o persona autorizada, con esTitularOAutorizado(vecino) — esta validación corresponde a la regla de negocio que establece que solo el titular o alguien autorizado puede reclamar.

El resultado de esa verificación se modela con un fragmento alt de dos ramas:

- Rama autorizado = true (flujo principal): el controlador determina la prioridad mediante un mensaje reflexivo (determinarPrioridad(tipo)), crea el Reclamo en estado ABIERTO y confirma la operación al vecino.
- Rama autorizado = false (flujo de error): el controlador no crea el reclamo y devuelve un mensaje de error, que la pantalla muestra al vecino.

PUNTO 5 - APLICACION DE PATRONES

Principios SOLID (3)

1. Responsabilidad Única (SRP)

- Problema detectado: si PantallaNuevoReclamo manejara validación, lógica de negocio y persistencia, tendría varias razones para cambiar: un cambio de UI, uno de reglas de reclamos y uno de almacenamiento tocarían la misma clase.
- Decisión: separar en tres capas con un solo motivo de cambio cada una: Pantalla (UI), Controlador, entidades de dominio (Reclamo, Expensa) que manejan su propio estado.
- Clases: PantallaNuevoReclamo / ControladorReclamos / Reclamo.
- Ventaja: mayor cohesión interna y cambios localizados.
- Relación GRASP: Alta Cohesión.

2. Abierto/Cerrado (OCP)

- Problema detectado: decidir la prioridad de una falla con condicionales por tipo de equipo obligaría a modificar ese código cada vez que se agregue un equipo nuevo.
- Decisión: Equipo es una clase abstracta con prioridadFalla(); cada subclase lo sobreescribe con su propia regla.
- Clases: Equipo (abstracta) → Luminaria, Camara.
- Ventaja: agregar un equipo nuevo es crear una subclase, sin tocar codigo existente.
- Relación GRASP: Polimorfismo y Variaciones Protegidas.

3. Sustitución de Liskov (LSP)

- Problema detectado: si Reclamo se asociara con clases concretas de vecino o de personal en vez de las clases base, agregar un nuevo tipo de persona rompería esa asociación.
- Decisión: Reclamo se asocia con Vecino y Personal (esta última abstracta). Cualquier subtipo de Personal puede sustituir a su superclase sin alterar el comportamiento esperado.
- Clases: Vecino; Personal → Administrador/Guardia/EmpleadoMantenimiento.
- Ventaja: el código cliente no necesita verificar el tipo concreto.
- Relación GRASP: Polimorfismo.

PATRONES GRASP

1. Experto en Información

- Problema detectado: ¿quién calcula el interés por mora de una expensa?
- Decisión: Expensa calcula su propio interés (calcularInteresMora()) porque tiene los datos necesarios: importe y fecha de vencimiento.
- Clases: Expensa.
- Ventaja: reduce acoplamiento y aumenta cohesión.

2. Controlador

- Problema detectado: si la pantalla invocara directamente a Lote o Reclamo, la UI quedaría acoplada al modelo de dominio.
- Decisión: introducir ControladorReclamos y ControladorExpensas como intermediarios entre la UI y las entidades.
- Clases: ambos controladores.
- Ventaja: la UI solo conoce al controlador; se podría cambiar de interfaz sin tocar la lógica de negocio.

3. Polimorfismo

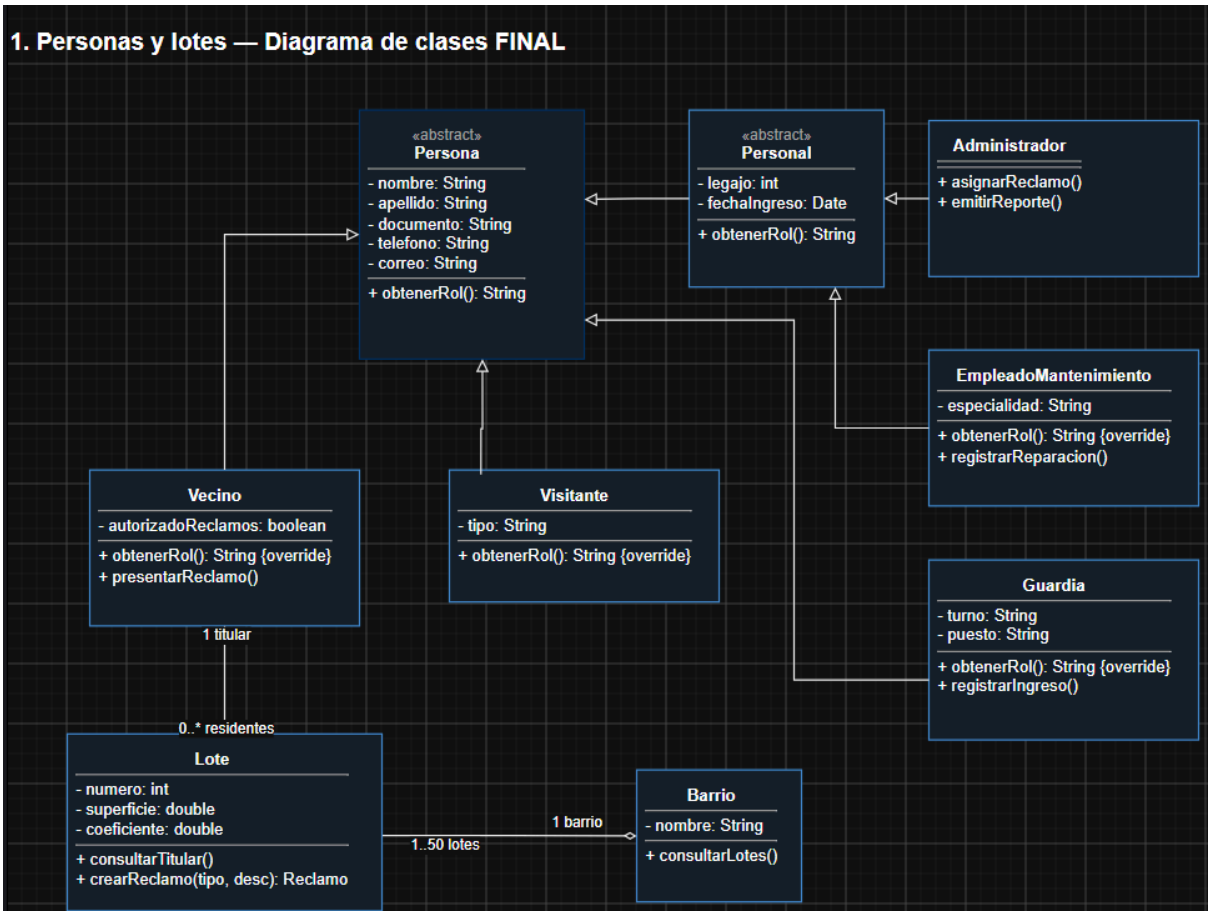
- Problema detectado: decidir la prioridad de falla de un equipo, o el rol de una persona, con condicionales por tipo sería frágil.
- Decisión: sobreescritura de método en cada subclase (prioridadFalla() en Equipo; obtenerRol() en Persona).
- Clases: Equipo, Persona, Personal.
- Ventaja: cada subclase resuelve su propio comportamiento sin condicionales externos.

4. Creador

- Problema detectado: en el diagrama de secuencia “Presentar reclamo”, ControladorReclamos crea directamente la instancia de Reclamo. Pero Lote tiene la asociación que agrupa los reclamos, así que debería ser el responsable de crearlos.
- Decisión: mover la creación a Lote.crearReclamo(tipo, desc); el controlador solo delega la orden.
- Clases: Lote y Reclamo.
- Ventaja: la lógica de creación queda junto a los datos que la agrupan.

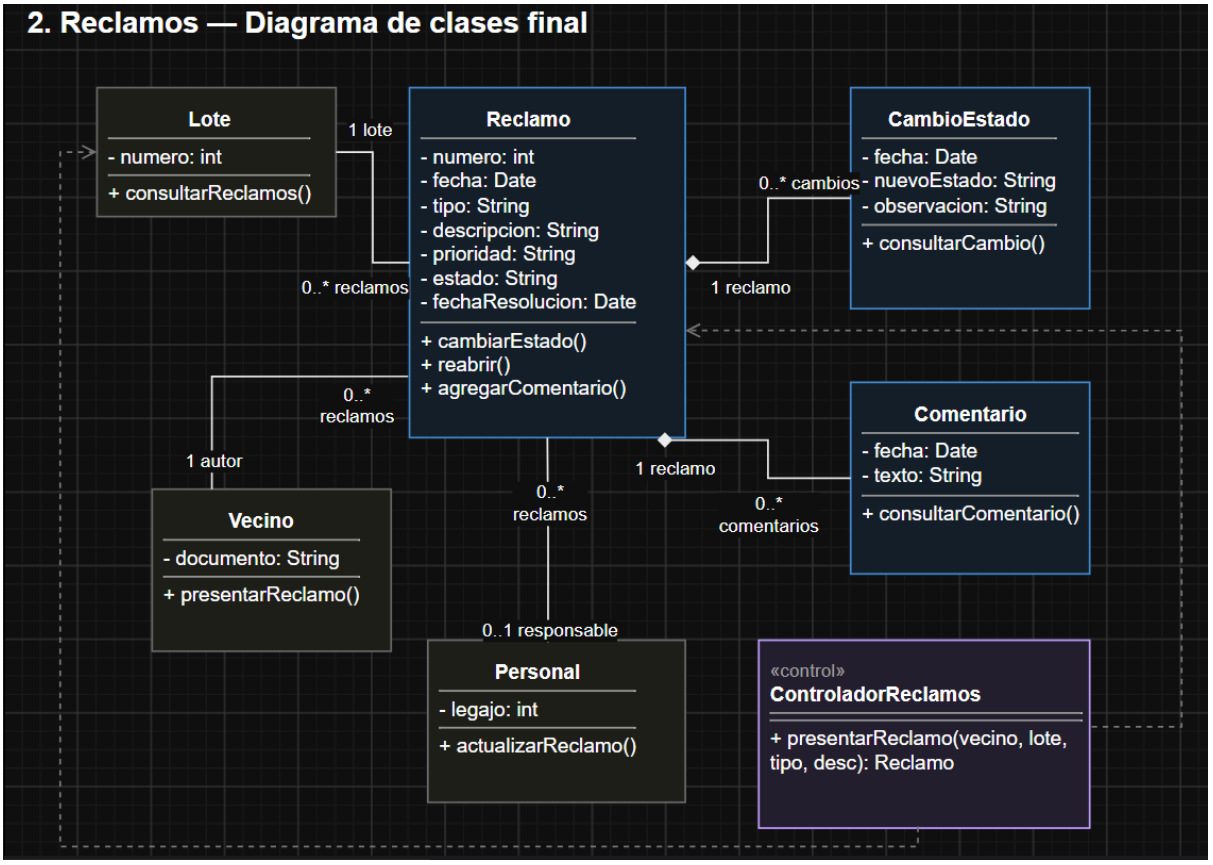
PUNTO 6 - DIAGRAMAS DE CLASES FINALES

1. personas y lotes



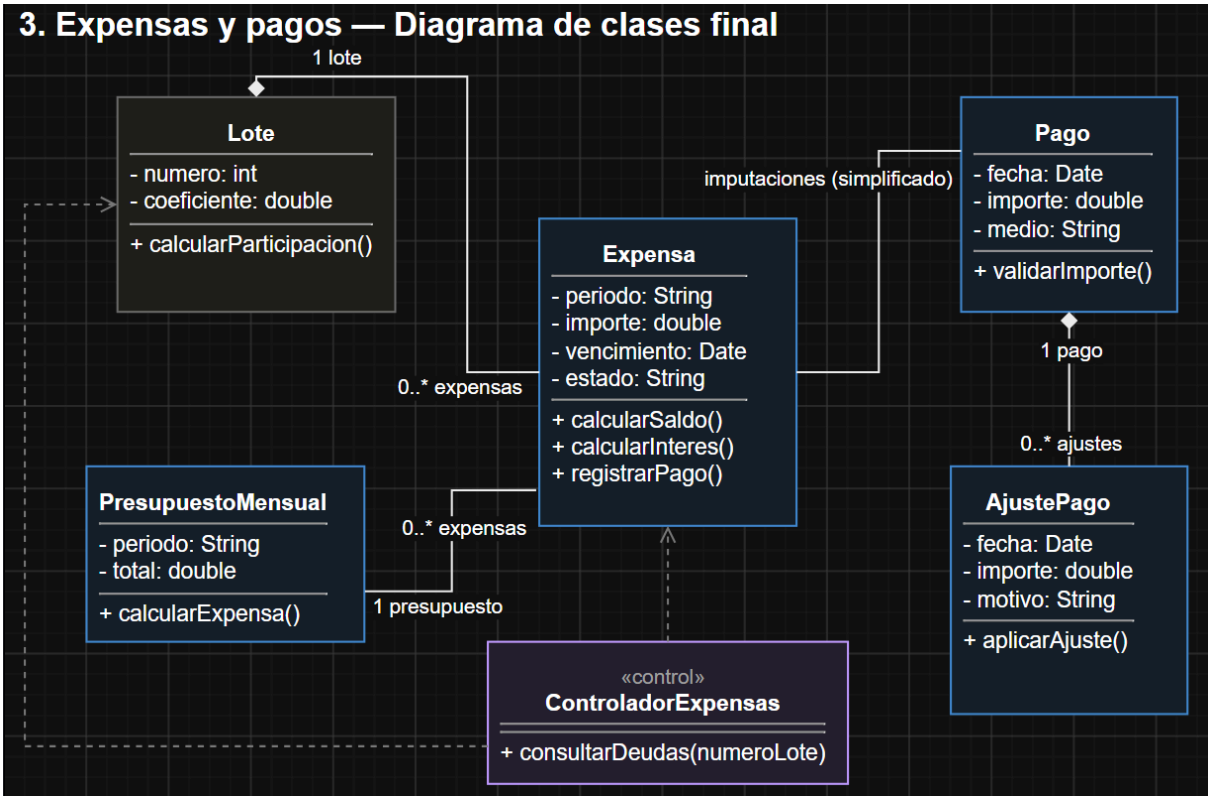
Cambio respecto de la version inicial: se agregó el metodo crearReclamo(tipo, desc) a Lote. En el diagrama de secuencia era el controlador quien creaba el Reclamo directamente; se trasladó a Lote por el patrón GRASP Creador, ya que Lote es quien agrupa los reclamos.

2. reclamos



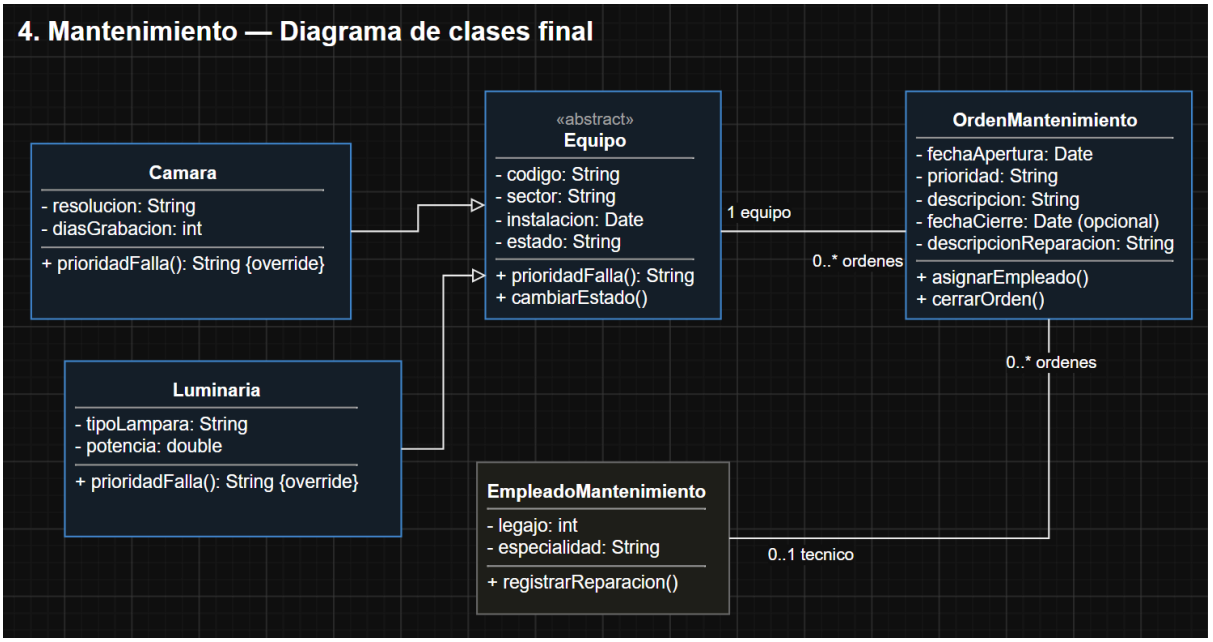
Cambio respecto de la version inicial: se agregó la clase ControladorReclamos, intermediaria entre la pantalla y las clases de dominio (Lote, Reclamo), que delega la creacion del reclamo en Lote.crearReclamo(). Aplica SRP (separar UI / control / dominio) y el patrón GRASP Controlador, segun el punto 5.

3. expensas y pagos



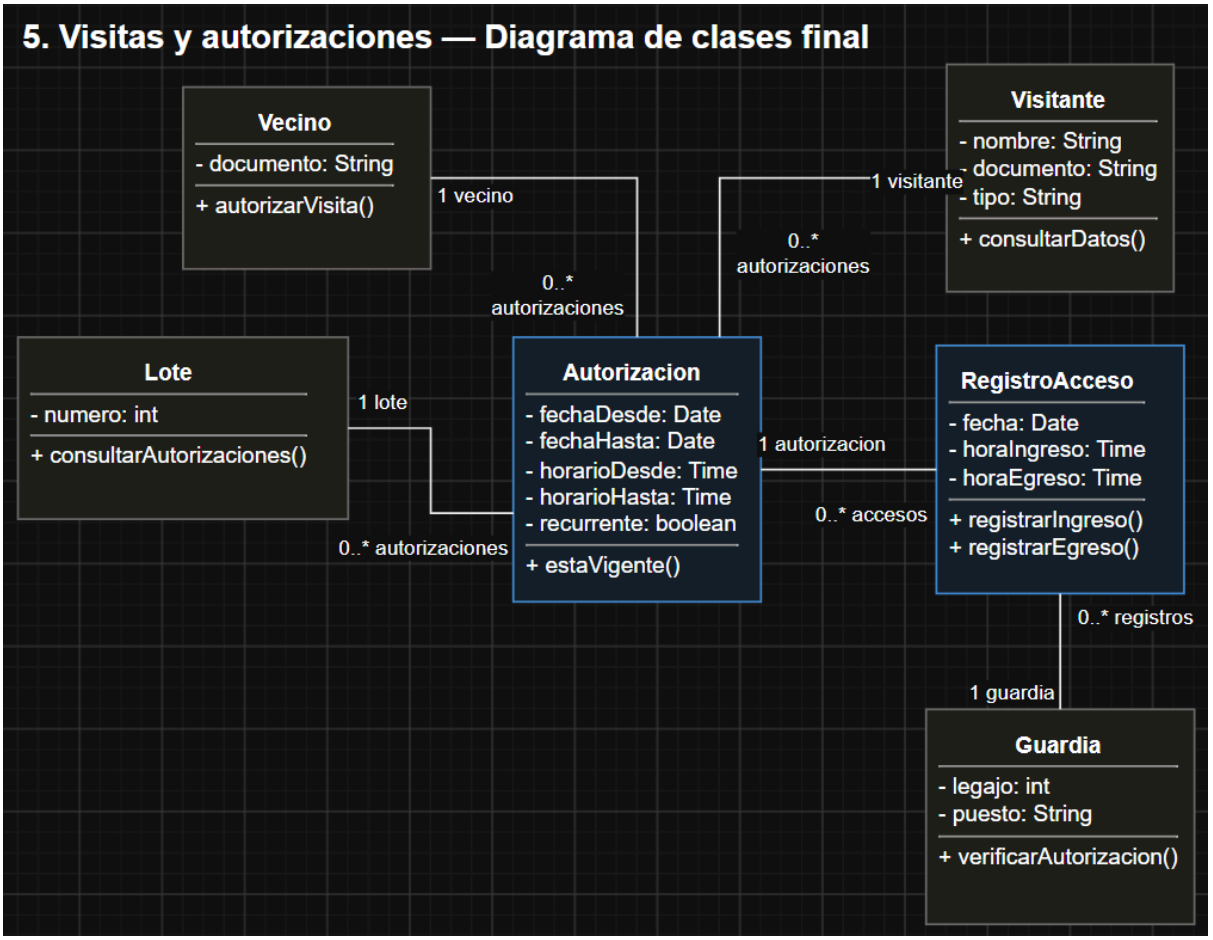
Cambio respecto de la version inicial: se agregó la clase ControladorExpensas, intermediaria entre la pantalla y las clases de dominio (Lote, Expensa), tal como se ve en el diagrama de secuencia "Consultar deudas". Aplica el patrón GRASP Controlador, segun el punto 5 (mismo criterio que ControladorReclamos en la sección de Reclamos).

4. mantenimiento



Sin cambios respecto de la version inicial: el diseño ya incorporaba desde el principio el principio OCP (Equipo como clase abstracta con prioridadFalla(), sobreescrito en Camara y Luminaria) y el patron GRASP Polimorfismo, segun el analisis del punto 5. No hizo falta modificar esta seccion.

5. visitas y autorizaciones



sin cambios respecto de la version inicial: el análisis del punto 5 no identificó ningun problema de diseño en esta sección (SRP/OCP/LSP, Experto, Controlador, Polimorfismo y Creador se aplicaron en Personas y lotes, Reclamos, Expensas y Mantenimiento). El diagrama de Visitas y Autorizaciones se mantiene igual que en la version inicial.